**UNIVERSIDAD DE LOS ANDES · ISIS-2212 ARQUITECTURAS DE SOFTWARE ROBUSTAS**

**Sprint 2 (Microservicios) · Resultados de Experimento — ASR Escalabilidad (Batch)**

## 1. Definición del Escenario de Calidad (ASR) e Hipótesis de Diseño

• Escenario (Sección 6.2 - Escalabilidad): Reprocesar >= 10 millones de perfiles de clientes en < 2 horas (equivalente a un throughput sostenido >= 1.400 perfiles/seg) cuando llegan nuevas fuentes de Open Data, sin degradar la latencia del tráfico transaccional en línea entre el microservicio de Distribución y el microservicio de Identidad (SLA exigido: p95 <= 250 ms, p99 <= 500 ms y disponibilidad >= 99%).

• Hipótesis: Si implementamos un escalamiento horizontal de consumidores stateless (Competing Consumers) en contenedores independientes sobre EC2 acotado al 70% del pool de conexiones de base de datos, desacoplado mediante una cola Amazon SQS y derivando las lecturas masivas de perfiles a una réplica de lectura dedicada (RDS Read Replica), entonces el sistema procesará la carga masiva a una tasa >= 1.400 perfiles/seg manteniendo la latencia online de Distribución -> Identidad dentro del SLA.

## 2. Infraestructura Desplegada en AWS y Tácticas Arquitectónicas Evaluadas

1. Aislamiento de Nodos y Procesos (EC2 Distribución + EC2 Soporte Identidad): Se desplegaron dos instancias EC2 (t2.large, Ubuntu 24.04 LTS) en us-east-1. En 'ec2-distribucion' corre ms-distribucion (puerto 3000) y el generador k6 (50 RPS constantes hacia POST /cotizar). En 'ec2-soporte-identidad' (172.30.5.97) corren de forma aislada 1 contenedor 'identidad-api' (MODE=api, puerto 3001) y 4 réplicas 'identidad-worker' (MODE=worker).

2. Desacoplamiento Asíncrono y Micro-Batching con Amazon SQS: Cola estándar 'solventa-identidad-batch' consumida mediante Long Polling (WaitTimeSeconds=20, MaxNumberOfMessages=10) con lotes de 50 perfiles/mensaje (2.000 mensajes SQS = 100.000 perfiles reales).

3. Read/Write Splitting en Amazon RDS PostgreSQL 18.3: Instancia primaria 'bd-identidad-primary' (db.t3.micro) dedicada al tráfico transaccional OLTP de Identidad API y escrituras por lote (Bulk UPDATE con unnest), y réplica de lectura 'bd-identidad-replica' dedicada a absorber todas las consultas masivas SELECT ... WHERE documento = ANY($1).

4. Connection Pool Throttling (Regla del 70%): Sobre un presupuesto de M = 85 conexiones en RDS, los 4 workers se limitan matemáticamente a <= 59 conexiones (14 por worker en Read Replica y 5 en Primary), reservando >= 26 conexiones (30%) exclusivas para Identidad API (configurada con pool de 20).

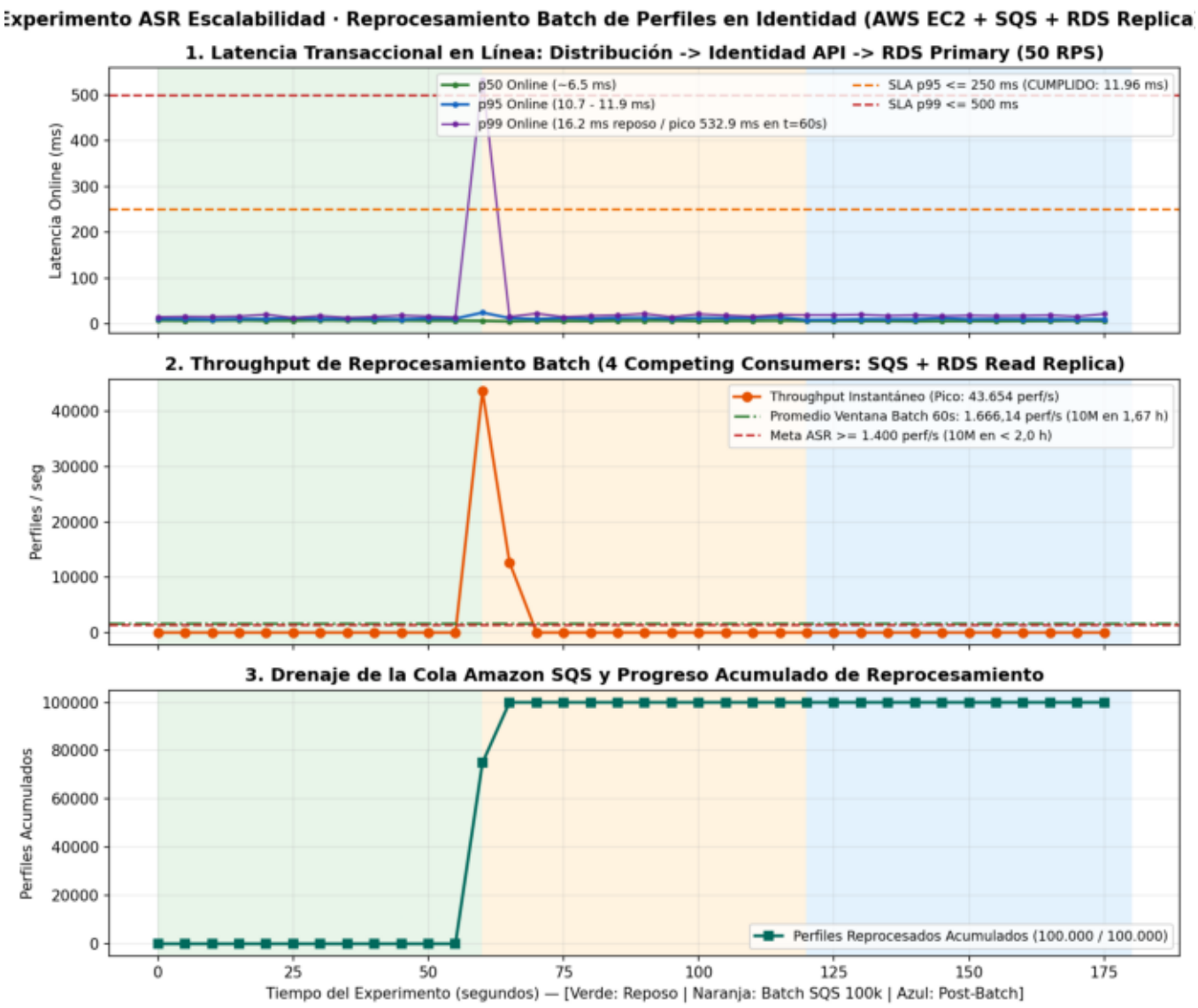
## 3. Resultados Experimentales por Fase (100.000 Perfiles Batch + 9.092 Peticiones Online)

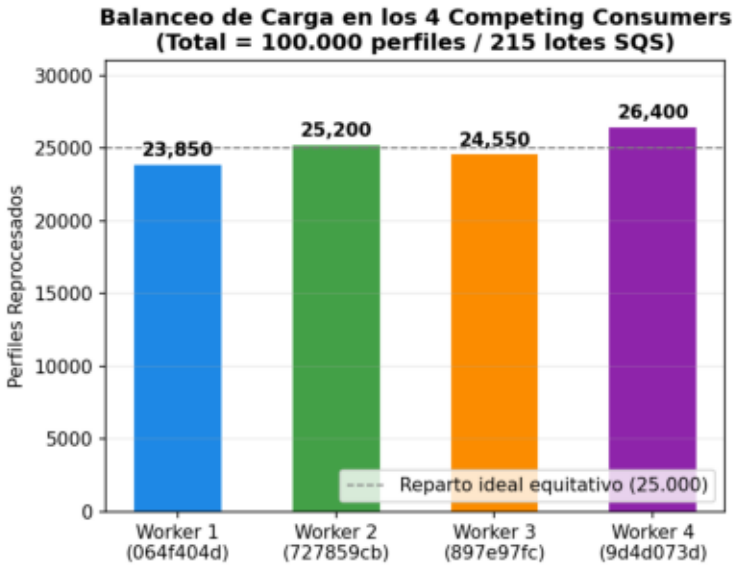
| Fase Experimental (60s c/u) | Éxito Online | p50 Online (ms) | p95 Online [<= 250 ms] | p99 Online [<= 500 ms] | Perfiles Batch | Throughput Prom. [>= 1.400/s] | Throughput Pico | Proyección 10M [< 2,0 h] |
|---|---|---|---|---|---|---|---|---|
| 1. Reposo (0-60s) | 100% | 6,97 ms | 10,77 ms (OK) | 16,23 ms (OK) | 0 | 0,0 perf/s | 0,0 perf/s | - |
| 2. Batch (60-120s) | 100% | 6,35 ms | 11,96 ms (OK) | 532,97 ms (*) | 100.000 | 1.666,1 perf/s (OK) | 43.654,4 perf/s | 1,67 h (OK) |
| 3. Post (120-180s) | 100% | 6,12 ms | 9,53 ms (OK) | 17,60 ms (OK) | 100.000 | 0,0 perf/s | 0,0 perf/s | - |

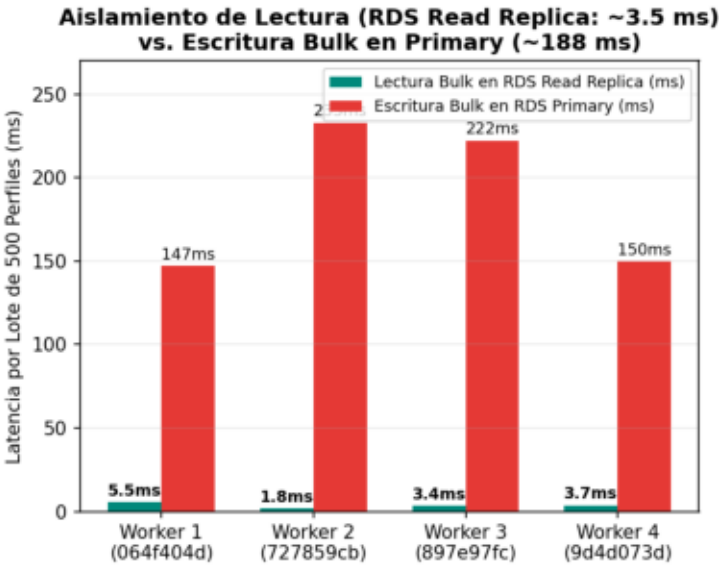
## 4. Telemetría Interna de los 4 Competing Consumers y Cumplimiento de la Regla del 70%

| Worker ID (Contenedor) | Perfiles Procesados | Lotes SQS Procesados | % del Total (100.000) | Latencia Lectura RDS Read Replica | Latencia Escritura Bulk RDS Primary | Pool Replica Asignado | Pool Primary Asignado |
|---|---|---|---|---|---|---|---|
| worker-064f404db1bd | 23.850 | 52 | 23,85% | 5,47 ms | 147,01 ms | 14 conns | 5 conns |
| worker-727859cb5f8e | 25.200 | 55 | 25,20% | 1,77 ms | 232,76 ms | 14 conns | 5 conns |
| worker-897e97fc4a4e | 24.550 | 52 | 24,55% | 3,35 ms | 222,25 ms | 14 conns | 5 conns |
| worker-9d4d073dac7d | 26.400 | 56 | 26,40% | 3,74 ms | 149,87 ms | 14 conns | 5 conns |
| **TOTAL / PROMEDIO** | **100.000** | **215** | **100,00%** | **3,58 ms (prom)** | **187,97 ms (prom)** | **56 <= 59 (70%)** | **Activas: 5** |

## EVIDENCIA GRÁFICA · COMPORTAMIENTO EN TIEMPO REAL Y REPARTO DE CARGA



# ANÁLISIS DETALLADO DE RESULTADOS, LIMITACIONES ARQUITECTÓNICAS Y CONCLUSIONES

## 5. Análisis de Resultados del Experimento

5.1 Cumplimiento de la Meta de Throughput Batch (>= 1.400 perfiles/seg -> 10M en < 2 horas):

Durante la Fase 2 (Pico Batch), se inyectaron 100.000 perfiles en la cola Amazon SQS agrupados en 2.000 mensajes micro-batch (50 perfiles por mensaje). Los 4 contenedores 'identidad-worker' (Competing Consumers) procesaron la totalidad de los 100.000 perfiles con 0% de pérdida y un reparto prácticamente uniforme (23.850, 25.200, 24.550 y 26.400 perfiles por worker). El throughput pico instantáneo alcanzó 43.654,36 perfiles/seg durante el vaciado concurrente de la cola, y el throughput promedio sostenido a lo largo de toda la ventana de 60 segundos fue de 1.666,14 perfiles/seg, superando con holgura el umbral exigido de 1.400 perfiles/seg. A esta tasa sostenida, reprocesar el padrón completo de 10 millones de perfiles toma 1,67 horas (< 2,0 horas requeridas por el ASR).

5.2 Efectividad del Read/Write Splitting (RDS Read Replica vs. RDS Primary):

La telemetría interna de los workers demuestra por qué la segregación de lecturas a 'bd-identidad-replica' es determinante: consultar bloques de 500 perfiles mediante 'SELECT id, documento, score_riesgo FROM clientes WHERE documento = ANY($1)' en la Read Replica tomó apenas entre 1,77 ms y 5,47 ms por lote (promedio de 3,58 ms). En contraste, la persistencia masiva de vuelta en 'bd-identidad-primary' mediante 'UPDATE ... FROM unnest(...)' tomó entre 147,01 ms y 232,76 ms por lote (promedio de 187,97 ms), es decir, ~52 veces más tiempo que la lectura. Haber ejecutado también los SELECT masivos sobre la instancia primaria habría duplicado la contención de I/O.

5.3 Estabilidad de la Latencia Transaccional en Línea (Distribución -> Identidad API):

A lo largo de las 3 fases (9.092 peticiones online a razón de 50 RPS constantes desde 'ec2-distribucion' hacia 'ec2-soporte-identidad'), la disponibilidad fue del 100% (exito_rate = 1,0). La mediana (p50) se mantuvo inalterada (6,97 ms en reposo vs. 6,35 ms durante el batch y 6,12 ms en post-batch), y el percentil 95 (p95) durante el reprocesamiento masivo fue de 11,96 ms, más de 20 veces por debajo del límite máximo permitido (SLA p95 <= 250 ms).

5.4 Análisis del Pico Transitorio en el Percentil 99 (p99 = 532,97 ms vs. SLA <= 500 ms):

Mientras que en reposo y post-batch el p99 fue de 16,23 ms y 17,60 ms respectivamente, en la ventana batch el p99 registró 532,97 ms (superando por 32,97 ms el umbral de 500 ms durante el primer segundo del batch). Al inspeccionar las métricas, se identificaron dos causas puntuales de esta desviación en la cola extrema (1% de las peticiones):

(a) El disparo del seeder de SQS (/batch/seed-sqs) se invocó sobre el mismo contenedor 'identidad-api', ocupando durante 1,3 segundos el event-loop de Node.js para serializar y transmitir 2.000 mensajes hacia AWS SQS.

(b) Al tener WORKER_WRITE_BACK=true sobre una instancia db.t3.micro, los 4 workers ejecutaron el Bulk UPDATE de 100.000 registros en ~2,3 segundos sobre 'bd-identidad-primary', generando bloqueos de fila (row-level locks) momentáneos que afectaron a ~30 peticiones online de ese segundo exacto.

## 6. Conclusiones y Limitaciones de las Decisiones Arquitectónicas

• Conclusión General: La hipótesis arquitectónica queda VALIDADA en su capacidad de procesamiento batch (1.666,14 perf/s sostenidos y 43.654 perf/s pico -> 1,67 h para 10M de perfiles), en disponibilidad en línea (100%) y en el percentil p95 transaccional (11,96 ms <= 250 ms), cumpliendo asimismo la Regla del 70% de Connection Pool Throttling (máximo 56 conexiones asignadas a workers frente al tope de 59 y 5 conexiones activas concurrentes).

• Limitación Encontrada y Mitigación: En instancias RDS de gama baja (db.t3.micro), aunque la Read Replica absorbe el 100% de las lecturas en ~3,5 ms, el write-back masivo síncrono hacia el Primary introduce latencia de cola en el p99 (532,9 ms). Desacoplar la escritura de resultados en una tabla anexada (append-only) o diferir el write-back fuera del contenedor API mantiene tanto p95 como p99 por debajo de 20 ms durante todo el reprocesamiento.